



1. Problema y oportunidad

Organizar gastos grupales —vaquitas para regalos, viajes, asados, eventos— es una práctica cotidiana en Argentina y en toda Latinoamérica. Sin embargo, la gestión de estos fondos es mayormente informal y carece de transparencia.

PROBLEMA IDENTIFICADO

Cuando un grupo junta dinero, siempre surge la misma fricción: "¿quién puso qué?", "¿cuánto gastamos en total?", "¿me podés mostrar el comprobante?". No existe un registro confiable que todos puedan verificar.

ALTERNATIVAS ACTUALES Y SU LIMITACIÓN

WhatsApp (informal, sin estructura), Splitwise (centralizado — el administrador puede editar los datos), hojas de cálculo (modificables por cualquiera con acceso).

Usuarios afectados: grupos de amigos, estudiantes universitarios, clubes, comunidades informales — cualquier colectivo que gestione fondos compartidos sin una entidad formal de respaldo.

Por qué vale la pena resolverlo: la desconfianza en la administración de fondos es la principal causa de conflictos en grupos. Una solución técnica que elimine la necesidad de confiar en el organizador resuelve el problema en la raíz.

2. Solución propuesta

Vaquitapp es una aplicación web que permite crear y gestionar grupos de ahorro digital ("vaquitas") con registro inmutable en blockchain. El historial de aportes y gastos queda grabado en un smart contract Solidity que nadie — ni el creador del grupo — puede modificar retroactivamente.

PRINCIPALES CARACTERÍSTICAS

- **Identidad descentralizada:** el usuario se autentica con su wallet MetaMask — sin email ni contraseña.
- **Creación de grupos:** nombre, descripción, categoría y objetivo de ahorro.
- **Sistema de roles:** Admin (crea, aprueba, carga gastos), Aprobado (puede aportar), Pendiente (espera aprobación).
- **Historial inmutable:** cada aporte y gasto genera un evento en la cadena, auditable públicamente.
- **Modo demo:** navegable sin MetaMask con datos mock — para evaluadores sin wallet configurada.

Diferencia clave vs. alternativas: el historial no puede ser alterado por nadie. La transparencia es una garantía técnica, no una promesa.

3. MVP desarrollado

✓ FUNCIONALIDADES IMPLEMENTADAS

- Conexión con MetaMask y autenticación por wallet
- Crear perfil (apodo + avatar emoji)
- Crear vaquita con nombre, categoría y objetivo
- Solicitar ingreso y aprobación de miembros
- Cargar aportes (VAQ\$)
- Cargar gastos con concepto y categoría
- Historial de movimientos con saldo en tiempo real
- Modo demo sin wallet (datos mock)
- 21 tests unitarios del contrato
- Deploy en Vercel (modo demo público)

→ SOON FUNCIONALIDADES FUTURAS (NO INCLUIDAS EN MVP)

- Invitaciones por link o código QR
- Notificaciones en tiempo real (eventos blockchain)
- Deploy en red pública (Polygon / Sepolia)
- Sistema de votación para gastos grandes
- IPFS para fotos de perfil
- Explorador público de vaquitas
- Soporte multi-red

URL demo pública: vaquitapp-gamma.vercel.app (modo demo, sin wallet requerida)

4. Componentes tecnológicos

STACK TECNOLÓGICO

Capa	Tecnología	Rol en el proyecto
Frontend	Next.js 14 · JavaScript · Pages Router	Interfaz web, rutas, SSR
Estilos	Tailwind CSS v3	UI responsiva sin CSS custom
Smart contract	Solidity 0.8.24	Lógica de negocio en blockchain
Entorno blockchain	Hardhat 2.22	Nodo local, compilación, tests, deploy
Integración web3	ethers.js v6	Comunicación frontend ↔ contrato
Wallet / auth	MetaMask (BrowserProvider)	Identidad y firma de transacciones
Datos auxiliares	localStorage	Apodo y avatar del usuario (off-chain)
Deploy frontend	Vercel	Hosting gratuito, CI/CD automático

INTELIGENCIA ARTIFICIAL — HERRAMIENTA DE DESARROLLO

CLAUDE CODE (ANTHROPIC) — IA GENERATIVA COMO PAIR PROGRAMMER

Durante el desarrollo se utilizó **Claude Code** (modelo claude-sonnet-4-6 de Anthropic) como asistente de programación con IA. Esta herramienta permitió:

- Generar y depurar el smart contract Solidity con validaciones de roles complejas
- Diseñar la arquitectura de integración ethers.js v6 con BrowserProvider y manejo de BigInt
- Implementar el modo demo con datos mock que replica el comportamiento del contrato real
- Generar los 21 tests unitarios con Hardhat y la suite de Chai
- Producir documentación (README, Lean Canvas, Pitch Deck) de forma asistida

Por qué Claude Code: permite iterar rápidamente en código complejo (Solidity, ethers.js v6) con validación inmediata — aceleró el desarrollo en un factor estimado de 3–4x respecto al desarrollo manual.

BLOCKCHAIN — DETALLE DE IMPLEMENTACIÓN

CONTRATO VAQUITAPP.SOL — SOLIDITY 0.8.24 · HARDHAT LOCAL · CHAINID 1337

Cómo fue utilizada: la blockchain actúa como base de datos inmutable para todos los eventos de negocio. El contrato almacena grupos (*structs Vaquita*), miembros con roles (*mapping address → Role*), aportes y gastos.

Qué problema resuelve: elimina la necesidad de un servidor central o una base de datos mutable. Los datos son públicos, auditables y permanentes — ningún actor puede modificarlos retroactivamente.

Evidencia de funcionamiento: 21 tests pasan en verde (Hardhat + Chai). Demo en vivo muestra transacciones reales firmadas por MetaMask y confirmadas en la cadena local.

INTEGRACIÓN CON CONTENIDOS DE LA MATERIA

Línea temática	Aplicación concreta en Vaquitapp
Blockchain y contratos inteligentes	Contrato Solidity con lógica de roles, aportes y gastos — nodo Hardhat local, deploy automatizado
Lenguajes de programación	Solidity (tipado estático, structs, mappings, eventos) + JavaScript (frontend dinámico, async/await)
Seguridad informática	Autenticación sin servidor (wallet), firma criptográfica de transacciones, validaciones de rol en el contrato
Sistemas de tipos	uint256 para montos, address para identidad, enum Role, BigInt conversion en ethers.js v6

ASPECTOS INNOVADORES

- Transparencia radical garantizada técnicamente (no por reputación)
- Identidad descentralizada — sin base de datos de usuarios
- Modo demo que replica blockchain con datos mock — UX idéntica
- Montos simbólicos (VAQ\$) que demuestran el concepto sin riesgo financiero

IMPACTO ESPERADO

- **Social:** reduce conflictos en grupos por falta de transparencia
- **Económico:** modelo freemium — gratuito en etapa de adopción
- **Ambiental:** red Hardhat local / futuro en Polygon PoS (bajo consumo)
- **Métrica de éxito:** vaquitas creadas / mes, tasa de conflictos reportados

6. Viabilidad y sustentabilidad

FACTIBILIDAD TÉCNICA

El MVP está funcionando. Stack probado: Next.js + Hardhat + ethers.js v6 son tecnologías maduras con comunidades activas. El único punto de riesgo es la UX de MetaMask para usuarios no técnicos — mitigable con el modo demo.

FACTIBILIDAD ECONÓMICA

Costo inicial: \$0 (Vercel gratuito, Hardhat local). En producción: gas fees en Polygon (~fracciones de centavo por tx), plan Vercel Pro (<\$20/mes). Ingresos: modelo freemium + comisiones simbólicas.

ESTRATEGIA DE CRECIMIENTO

Fase 1 (actual): MVP demo en Hardhat local. **Fase 2:** deploy en Polygon Mainnet, onboarding con QR. **Fase 3:** plan premium (organizaciones, clubs, empresas). **Aliados potenciales:** proyectos DeFi, wallets web3, exchanges regionales.

7. Trabajo en equipo

El proyecto fue desarrollado en equipo aplicando metodología ágil con iteraciones cortas. Las decisiones se tomaron en conjunto priorizando la demo funcional sobre la completitud de features.

Integrante	Rol principal	Responsabilidades
Gaston Miranda	Blockchain / Backend	Smart contract Solidity, tests Hardhat, scripts de deploy
Alejo Valls	Frontend / Integración	Next.js, ethers.js, UI Tailwind, modo demo, documentación

METODOLOGÍA

- Repositorio compartido en GitHub con commits frecuentes
- Backlog priorizado: contrato → tests → frontend → modo demo → documentación
- Revisión de código entre pares antes de cada merge

8. Aprendizajes y propuestas descartadas

IDEAS DESCARTADAS Y POR QUÉ

- **IPFS para avatares:** complejidad innecesaria para el MVP — localStorage suficiente
- **Deploy en Sepolia/Polygon:** requiere ETH/MATIC real — fuera del alcance de demo universitaria
- **TypeScript:** agregaría tiempo de setup sin beneficio visible en una demo de este tamaño
- **Base de datos SQL:** contradice el principio de descentralización de la propuesta

APRENDIZAJES OBTENIDOS

- ethers.js v6 rompe compatibilidad con v5 — BigInt vs Number requiere conversión explícita
- Un modo demo bien implementado es tan valioso como la integración real para evaluadores
- Los smart contracts no tienen estado mutable libre — diseñar bien los structs desde el inicio evita migraciones
- La UX de MetaMask es una barrera real: el modo demo fue clave para la accesibilidad

9. Fuentes y referencias

- Documentación oficial de Solidity — docs.soliditylang.org
- Documentación de ethers.js v6 — docs.ethers.org/v6
- Documentación de Hardhat — hardhat.org/docs
- Next.js 14 Pages Router — nextjs.org/docs
- MetaMask Developer Docs — docs.metamask.io
- Ethereum Yellow Paper — Gavin Wood, ethereum.github.io/yellowpaper
- Claude Code (Anthropic) — asistente de IA utilizado en el desarrollo — anthropic.com